



User Permissions and File System on Linux

17-12-2025

! Why This Topic Matters (Seriously)

Linux doesn't break because it's fragile. Linux breaks because **permissions exist** and **you don't understand them yet**.

Permissions:

- Protect your system
- Prevent malware from wrecking everything
- Make Linux powerful for servers, DevOps, security, and cloud

Once this clicks:

- Errors make sense
- "Permission denied" stops ruining your day
- You stop nuking your system with `sudo rm -rf`

1 🌲 Linux File System Hierarchy (The Big Picture)

Linux doesn't use drive letters like Windows (C:\, D:\). Instead, **everything starts at one place**:

/ — The Root of Everything

/ is the **top of the file system tree**.

Think of it like:

The trunk of a tree 🌲
Everything grows from here.

```
$ /  
  home  
  etc  
  var  
  usr  
  bin  
  sbin  
  tmp
```

⚠️ You **do not** casually mess around in /.

/home — Where Humans Live

This is where **users live**.

```
$ /home/chuck  
$ /home/student
```

Inside your home directory:

- Documents
- Downloads
- Config files
- SSH keys
- Hidden dot files

Your home is:

- Owned by you
- Writable by you
- Mostly safe from system damage

👍 Best Practice:

If you're learning Linux, live in /home

/etc — Configuration Central

This is one of the **most important directories**.

/etc contains:

- System configuration files
- User settings
- Service configs
- Network configs

Examples:

```
$ /etc/passwd  
$ /etc/shadow  
$ /etc/hosts  
$ /etc/sudoers
```

 Editing files here can:

- Break login
- Break networking
- Lock you out completely

 **Rule:**

/etc = configuration, not programs

/var — Variable Data (Stuff That Changes)

This is where Linux stores:

- Logs
- Caches
- Spools
- Databases

Examples:

```
$ /var/log
$ /var/log/syslog
$ /var/log/auth.log
```

👉 If a system is acting weird:

👉 **Try Checking** `/var/log` or Any other related logs for your problem

`/usr` — User Programs (Not Users)

Confusing name. This is **not** user home directories.

`/usr` contains:

- Installed programs
- Libraries
- Shared resources

```
$ /usr/bin
$ /usr/lib
$ /usr/share
```

Most software you install ends up here.

2 📁 "Everything Is a File" (Linux Philosophy)

This is where Linux becomes *cool*.

In Linux:

- Files
- Directories
- Devices
- Hardware
- Network interfaces

👉 **Everything is a file**

Examples:

```
$ /dev/sda # hard drive
$ /dev/null # black hole
$ /proc/cpuinfo # CPU info
$ /dev/tty # current terminal
$ /dev/random # random number generator
$ /dev/urandom # faster random number generator
$ /proc/uptime # system uptime
$ /proc/net/tcp # TCP connections
$ /sys/class/net/eth0/address # network MAC address
$ /sys/class/backlight/brightness # screen brightness
$ /sys/class/power_supply/BAT0/capacity # battery level
$ /dev/loop0 # loopback device (mount files as disks)
```

Want hardware info?

```
$ cat /proc/cpuinfo
```

Want to silence output?

```
$ command > /dev/null
```

👍 This design is WHY:

- Linux is scriptable
- Linux is automatable
- Linux runs the internet

3 Dot Files (Hidden Power)

Files that start with a dot . are **hidden**.

Example:

```
$ .bashrc
$ .profile
$ .gitconfig
$ .ssh/
```

They usually store:

- Shell configs

- App preferences
- User settings

List them:

```
$ ls -a
```

Why hide them?

- Prevent clutter
- Prevent accidental deletion

👉 **Important:**

Deleting dot files can reset your environment or break tools.

4 👤 File Ownership (Who Owns What?)

Every file has:

- **Owner**
- **Group**
- **Permissions**

Check with:

```
$ ls -l
```

Example:

```
$ -rwxr-xr-- 1 ahmad staff script.sh
```

Breakdown:

```
Owner: ahmad  
Group: staff
```

Ownership matters because:

- Owners control permissions
- Groups allow shared access
- Others are everyone else

Changing Ownership

```
$ chown user file  
$ chown user:group file
```

Example:

```
$ sudo chown root:root config.conf
```

⚠ Wrong ownership = broken apps

5 Permissions (rwx) — The Core Skill

Permissions are **everything**.

Three types:

- **r** = read
- **w** = write
- **x** = execute

Three levels:

- Owner
- Group
- Others

Example:

```
$ -rwxr-xr--
```

Break it down:

```
Owner: rwx  
Group: r-x  
Others: r--
```

Permission Numbers (Octal Mode)

This is where beginners panic—but don't.

Values:

```
r = 4  
w = 2  
x = 1
```

Add them:

```
7 = rwx  
6 = rw-  
5 = r-x  
4 = r--
```

Example:

```
$ chmod 755 script.sh
```

Meaning:

```
Owner: 7 (rwx)  
Group: 5 (r-x)  
Others: 5 (r-x)
```

👍 You will see 755 and 644 everywhere.

Making a Script Executable

```
$ chmod +x script.sh
```

Without execute permission:

```
$ ./script.sh  
# Permission denied
```

6 Root User (Unlimited Power)

Root is:

- The system administrator
- The power mode user
- The "I can change everything" account

Root can:

- Read any file
- Delete any file
- Change any permission

sudo — Temporary Root

```
$ sudo command
```

This:

- Executes command as root
- Logs actions
- Requires password

Best practice:

Use sudo, not permanent root login

su — Switch User

```
$ su
$ su username
```

This:

- Switches shell to another user
- Requires target user password

 **Dangerous if misused**

/etc/sudoers — Who Can Use sudo

DO NOT edit directly.

Always use:

```
$ sudo visudo
```

Example entry:

```
ahmad ALL=(ALL) ALL
```

⚠ Wrong syntax here => no sudo => bad day.

7 🔗 Hard Links vs Soft Links (Symbolic Links)

Links are **references** to files.

Hard Links

- Point to the same inode
- File exists in multiple locations
- Deleting one doesn't remove data

```
$ ln file1 file2
```

Limitations:

- Same filesystem only
- Can't link directories (usually)

Soft Links (Symlinks)

- Pointer to file path
- Like Windows shortcuts

```
$ ln -s target linkname
```

Example:

```
$ ln -s /var/log/syslog syslog_link
```

If target is deleted:

⚠ Link breaks

👍 Most commonly used type.

8 ⚠ Common Beginner Mistakes (Read This Twice)

This section will save your system.

⚠ Running Everything with sudo

Bad:

```
$ sudo npm install
$ sudo python script.py
```

Why bad?

- Files owned by root
- Permission nightmares
- Security risk

👍 Use sudo **only when required**

⚠ chmod 777 Everything

```
$ chmod 777 file
```

This means:

Everyone can do everything

Bad for:

- Security
- Best practices
- Your future job interviews 😊

⚠️ Editing System Files Without Backup

Before:

```
$ sudo cp config.conf config.conf.bak
```

Always.

⚠️ Deleting Random Stuff in /etc or /usr

If you don't know what it does: 🖱️ **Don't delete it**

⚠️ Not Reading Error Messages

Linux tells you:

- What failed
- Why it failed
- What permission is missing

Read it.

9 🖱️ Real-World Permission Scenarios

- **"Permission denied" when running script**
 - Possible Missing execute bit
- **App won't start**
 - Maybe Wrong ownership in config directory
- **Can't write to directory**
 - Check group permissions
- **Service won't start**
 - Maybe Config owned by wrong user

10 🐧 Final Linux Mindset Shift

Linux is not hard.

Linux is:

- Honest
- Strict
- Predictable

Permissions are not obstacles. They are **guardrails**.

Once you understand:

- File hierarchy
- Ownership
- Permissions
- Root access

You unlock:

- Servers
- DevOps
- Security
- Cloud
- Automation

What You Should Practice Next

- ✓ `ls -l` everywhere
- ✓ `chmod` with numbers
- ✓ `chown` safely
- ✓ Explore `/etc` (read-only)
- ✓ Break things **inside a VM**

If you want, next we can:

- Do a **hands-on lab**
- Build a **permission challenge**
- Simulate **real server mistakes**
- Or connect this to **Docker & servers**

Now go grab that coffee ☕
You just leveled up in Linux. 🐧🔥